

RELATÓRIO TÉCNICO: Sistema de Cadastro Escolar

Disciplina: Introdução às Técnicas de Programação

Unidade: 2

Aluno: Breno Igor da Silva

Matrícula: 20250065477

Período: 2025.2

1 INTRODUÇÃO E CONTEXTO

1.1 Nome do Projeto e Evolução

O Sistema de Cadastro Escolar foi expandido na U2 com novos conceitos de programação. O sistema continua gerenciando matrículas de alunos do ensino fundamental, mas agora conta com funcionalidades mais avançadas de busca, visualização de dados e gerenciamento de memória.

1.2 Objetivos da U2

Os objetivos primários desta unidade foram a aplicação prática de estruturas de dados e otimização de recursos:

- Implementar sistema de busca avançada usando strings.
- Criar matriz bidimensional para visualização de dados.
- Aplicar ponteiros para otimizar passagem de dados.
- Utilizar loops aninhados para estruturas bidimensionais.

1.3 Funcionalidades Implementadas

As seguintes funcionalidades foram adicionadas ou aprimoradas na Unidade 2:

- Busca por nome completo usando `strcmp`.
- Busca por série e turma específica.
- Busca por turno organizada por série.
- Matriz de distribuição de alunos por série e turma.

2 ANÁLISE TÉCNICA

2.1 Metodologia - Ferramentas Utilizadas

- **Sistema Operacional:** Linux
- **Compilador:** GCC versão 13.3.0
- **Editor de Código:** Visual Studio Code
- **Linguagem:** C

2.2 Aplicação dos Conceitos da U2

2.2.1 Strings

A função `strcmp()` da biblioteca `string.h` foi implementada para realizar a comparação exata de strings na funcionalidade de busca por nome. Esta função retorna 0 quando as strings comparadas são idênticas, garantindo a precisão da busca.

```
if(strcmp(lista[i].nome, nome) == 0){  
    // Aluno encontrado  
}
```

Arrays de char continuam sendo utilizados para armazenar nomes (até 50 caracteres) e dados dos responsáveis. A leitura de linhas completas no `scanf` foi mantida utilizando o formato `%49[^\n]`.

2.2.2 Estruturas de Repetição Aninhadas

Loops aninhados foram empregados para o tratamento eficiente de estruturas bidimensionais e fluxos de dados complexos, aplicados em três contextos principais:

- **Matriz de distribuição:** O loop externo itera sobre as séries, enquanto o loop interno percorre as turmas (A e B).
- **Busca por turno:** O primeiro loop itera sobre as séries, e o segundo loop itera sobre os alunos pertencentes àquela série.
- **Atualização da matriz:** Percorre a lista completa de alunos para, então, atualizar as posições correspondentes na matriz de distribuição.

2.2.3 Matrizes

A estrutura `distribuicao_turmas[MAX_SERIES][MAX_TURMAS]` representa um array bidimensional com a seguinte organização:

- **Linhas (0-8):** Correspondem às séries (1ª a 9ª).
- **Colunas (0-1):** Correspondem às turmas (A e B).
- **Células:** Armazenam a quantidade de alunos matriculados em cada combinação de série e turma.

As operações realizadas sobre esta matriz incluem inicialização, atualização automática após novos cadastros, exibição em formato tabular e totalização de alunos por linha e coluna. O acesso aos elementos utiliza a lógica de: índice da série é *serie - 1*, e o índice da turma é 0 para A e 1 para B.

2.2.4 Ponteiros

O conceito de ponteiros foi aplicado nas funções de busca, que recebem o array de estruturas Aluno como um ponteiro:

```
void buscar_por_nome(Aluno *lista, int total)
```

Essa abordagem garante que apenas o endereço de memória do array (o ponteiro) seja passado para a função, em vez de copiar o array inteiro. Isso resulta em otimização de uso de memória e potencial ganho de velocidade no acesso aos dados. Dentro das funções, a sintaxe `lista[i]` é mantida para acessar os elementos, aproveitando a aritmética de ponteiros implícita no C.

3 IMPLEMENTAÇÃO E REFLEXÃO

3.1 Dificuldades Encontradas

A principal dificuldade técnica encontrada foi a compreensão e aplicação de ponteiros, um conceito que ainda não havia sido profundamente estudado:

1. **Entender ponteiros:** A complexidade inicial residiu em assimilar o funcionamento dos ponteiros e em definir a forma mais clara e útil de implementá-los no projeto.

3.2 Soluções Implementadas

Para superar a dificuldade com ponteiros, foram adotadas as seguintes estratégias:

1. **Para ponteiros:** Foi realizada prática intensa com a linguagem C, o que levou à percepção de que arrays em C são, por natureza, ponteiros. A inclusão de comentários detalhados no código fonte foi crucial para internalizar e explicar a lógica de endereçamento e desreferência.

3.3 Organização do Código

O código foi estruturado em módulos lógicos para facilitar a manutenção e a legibilidade, seguindo a ordem:

- Definições (`#define`) e estruturas (`struct`) no topo do arquivo.
- Variáveis globais (incluindo o ponteiro para a lista de alunos).
- Funções de busca e manipulação de strings (novas da U2).
- Funções de manipulação da matriz de distribuição.
- Funções originais de cadastro e exibição da U1.
- Função principal `main` com o menu de interação.

3.4 Conclusão

A Unidade 2 proporcionou um avanço significativo na compreensão de estruturas de dados e técnicas de otimização em C.

Aprendizados obtidos:

- A aplicação prática de ponteiros na passagem de arrays de estruturas para funções clarificou seu poder em economia de memória e agilidade de acesso.
- O trabalho com matrizes demonstrou, na prática, a organização intuitiva e visual de dados complexos, como a distribuição de alunos por série e turma.
- A necessidade de manipular estruturas bidimensionais tornou o uso de loops aninhados um processo mais natural e fundamental para a arquitetura do sistema.

O projeto está pronto para receber novas expansões, como a persistência de dados em arquivos ou a implementação de árvores de busca.